

Database System Engineering and Distributed Backend Development

Course Code: 25CS1302E

Week -10

STEP 1 — Create FastAPI Project Folder

▼ FASTAPI_HTTP_METHODS

STEP 2 — Create Virtual Environment

```
vsameerreddy@Sameer-Reddy fastapi % cd ~/Desktop/CollegeResearchProject/fastapi
vsameerreddy@Sameer-Reddy fastapi % source venv/bin/activate
(venv) vsameerreddy@Sameer-Reddy fastapi % uvicorn main:app --reload --port 8000
INFO: Will watch for changes in these directories: ['/Users/vsameerreddy/Desktop/CollegeResearchProject/fastapi']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

STEP 3 — Install FastAPI and Uvicorn

Run:

```
python -m pip install --upgrade pip
```

Then:

```
python -m pip install fastapi uvicorn email-validator
```

Wait until installation finishes.

STEP 4 — Create requirements.txt

Run:

pip freeze > requirements.txt

STEP 5 — Create main.py

In the VS Code Explorer:

Right-click project folder → New File

Name it:

main.py

and paste

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, EmailStr

app = FastAPI(
    title="Student Management API",
    description="A practical FastAPI project demonstrating HTTP methods",
    version="1.0.0"
)

class Student(BaseModel):
    name: str
    age: int
    course: str
    email: EmailStr

students = {}

@app.get("/")
def home():
    return {"message": "Welcome to Student Management API"}

@app.get("/students")
def get_students():
    return {"students": students}

@app.get("/students/{student_id}")
def get_student(student_id: int):
    if student_id not in students:
        raise HTTPException(status_code=404, detail="Student not found")
    return students[student_id]

@app.post("/students", status_code=201)
def create_student(student_id: int, student: Student):
    if student_id in students:
        raise HTTPException(status_code=400, detail="Student ID already exists")
    students[student_id] = student.model_dump()
    return {
        "message": "Student created successfully",
        "student_id": student_id,
        "student": students[student_id]
    }

@app.put("/students/{student_id}")
def update_student(student_id: int, student: Student):
    if student_id not in students:
        raise HTTPException(status_code=404, detail="Student not found")
```

```

students[student_id] = student.model_dump()
return {
    "message": "Student completely updated",
    "student_id": student_id,
    "student": students[student_id]
}

class StudentUpdate(BaseModel):
    name: str | None = None
    age: int | None = None
    course: str | None = None
    email: EmailStr | None = None

@app.patch("/students/{student_id}")
def partial_update_student(student_id: int, student: StudentUpdate):
    if student_id not in students:
        raise HTTPException(status_code=404, detail="Student not found")
    update_data = student.model_dump(exclude_unset=True)
    students[student_id].update(update_data)
    return {
        "message": "Student partially updated",
        "student_id": student_id,
        "student": students[student_id]
    }

@app.delete("/students/{student_id}")
def delete_student(student_id: int):
    if student_id not in students:
        raise HTTPException(status_code=404, detail="Student not found")
    deleted_student = students.pop(student_id)
    return {
        "message": "Student deleted successfully",
        "student_id": student_id,
        "student": deleted_student
    }

```

STEP 8 — Start the FastAPI Server

```

(venv) sahasra@sah fastapi_http_methods % python -m uvicorn main:app --reload
INFO: Will watch for changes in these directories: ['/Users/sahasra/DBMS /fastapi_http_meth
ods']
INFO: Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
INFO: Started reloader process [15866] using StatReload
INFO: Started server process [15868]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:62315 - "GET / HTTP/1.1" 200 OK

```

STEP 9 — Open Swagger UI

Student Management API

openapi.json

GET / Home

Parameters

No parameters

Execute Clear

Responses

Curl

Request URL

Server response

Code Details

200

Response body

Download

Schemas

HTTPValidationError > Expand all object

Student > Expand all object

StudentUpdate > Expand all object

STEP 10 — Test GET Root

Endpoint

STEP 11 — Test POST Create Student

POST /students Create Student

Parameters

Cancel Reset

Name

Description

student_id * required integer (query) 101

Request body * required application/json

Edit Value Schema

```
{  "name": "Rahul",  "age": 20,  "course": "Computer Science",  "email": "rahulgexample.com"}
```

Execute Clear

STEP 12 — Test GET All Students

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/students' \
  -H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/students
```

Server response

Code	Details
200	Response body <pre>{ "students": { "101": { "name": "Rahul", "age": 20, "course": "Computer Science", "email": "rahul@example.com" } } }</pre> Response headers <pre>content-length: 102 content-type: application/json date: Sat, 26 Sep 2026 03:34:05 GMT server: uvicorn</pre>

STEP 13 — Test
GET Student by

ID

GET /students/{student_id} Get Student

Parameters

Name	Description
student_id • required	101
integer	
(path)	

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/students/101' \
  -H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/students/101
```

Server response

Code	Details
200	Response body <pre>{ "name": "Rahul", "age": 20, "course": "Computer Science", "email": "rahul@example.com" }</pre>

STEP 14 — Test PUT Complete Update

Curl

```
curl -X 'PUT' \
'http://127.0.0.1:8000/students/101' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
  "name": "Rahul Kumar",
  "age": 21,
  "course": "Artificial Intelligence",
  "email": "rahulkumar@example.com"
}'
```

Request URL

<http://127.0.0.1:8000/students/101>

Server response

Code	Details
200	Response body <pre>{ "message": "Student completely updated", "student_id": 101, "student": { "name": "Rahul Kumar", "age": 21, "course": "Artificial Intelligence", "email": "rahulkumar@example.com" } }</pre>  Download

STEP 15 — Test PATCH Partial Update

Curl

```
curl -X 'PATCH' \
'http://127.0.0.1:8000/students/101' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
  "course": "Data Science"
}'
```

Request URL

<http://127.0.0.1:8000/students/101>

Server response

Code	Details
200	Response body <pre>{ "message": "Student partially updated", "student_id": 101, "student": { "name": "Rahul Kumar", "age": 21, "course": "Data Science", "email": "rahulkumar@example.com" } }</pre>  Download

STEP 16 — Test DELETE Student

Curl

```
curl -X 'DELETE' \
'http://127.0.0.1:8000/students/101' \
-H 'accept: */*' \
-H 'Content-Type: application/json'
```

Request URL

<http://127.0.0.1:8000/students/101>

Server response

Code	Details
200	Response body <pre>{ "message": "Student deleted successfully", "student_id": 101, "student": { "name": "Rahul Kumar", "age": 21, "course": "Data Science", "email": "rahulkumar@example.com" } }</pre>  Download

STEP 17 — Verify Student Deletion

Name	Description
student_id * required integer (path)	101

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/students/101' \
-H 'accept: */*' \
```

Request URL

```
http://127.0.0.1:8000/students/101
```

Server response

Code	Details
404 <i>Undocumented</i>	Error: Not Found Response body <pre>{ "detail": "Student not found" }</pre> Download

STEP 18 — Test Duplicate Student ID Error

STEP 19 — Test Missing Student Error

Curl

```
curl -X 'POST' \
'http://127.0.0.1:8000/students?student_id=101' \
-H 'accept: */*' \
-H 'Content-Type: application/json' \
-d '{
  "name": "Rahul",
  "age": 20,
  "course": "Computer Science",
  "email": "rahul@example.com"
}'
```

Request URL

```
http://127.0.0.1:8000/students?student_id=101
```

Server response

Code	Details
400 <i>Undocumented</i>	Error: Bad Request Response body <pre>{ "detail": "Student ID already exists" }</pre> Download

GET /students/{student_id} Get Student

Parameters Cancel

Name	Description
student_id * required	
integer	
(path)	

999

Execute Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/students/999' \
  -H 'accept: */*' \
```

Request URL

http://127.0.0.1:8000/students/999

Server response

Code	Details
404	Error: Not Found

Undocumented

Response body

```
{
  "detail": "Student not found"
}
```

Download

STEP 20 — Test Invalid Age

Validation

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/students?student_id=102' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Test Student",
    "age": "twenty",
    "course": "Computer Science",
    "email": "test@example.com"
  }'
```

Request URL

http://127.0.0.1:8000/students?student_id=102

Server response

Code	Details
422	Error: Unprocessable Content

Response body

```
{
  "detail": [
    {
      "type": "int_parsing",
      "loc": [
        "body",
        "age"
      ],
      "msg": "Input should be a valid integer, unable to parse string as an integer",
      "input": "twenty"
    }
  ]
}
```

Download

STEP 21 — Test Invalid Email Validation

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/students?student_id=103' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Test Student",
    "age": 20,
    "course": "Computer Science",
    "email": "invalid-email"
  }'
```

Request URL

http://127.0.0.1:8000/students?student_id=103

Server response

Code	Details
------	---------

422	Error: Unprocessable Content
-----	------------------------------

Response body

```
{
  "detail": [
    {
      "type": "value_error",
      "loc": [
        "body",
        "email"
      ],
      "msg": "value is not a valid email address: An email address must have an @-sign.",
      "input": "invalid-email",
      "ctx": {
        "reason": "An email address must have an @-sign."
      }
    }
  ]
}
```



Download

